S

SEPHIROTH · XRAY CONTROL PANEL

The Complete Sephiroth

CATALOG · v2.6.0

7 Aban 1405 — October 29, 2026

The official illustrated catalog of the next-generation Xray management panel — from the config Forge, Autopilot and AmneziaWG to user lifecycle, the notification engine, white-label, the Public API and the full install suite. Everything with real screenshots of the panel itself.

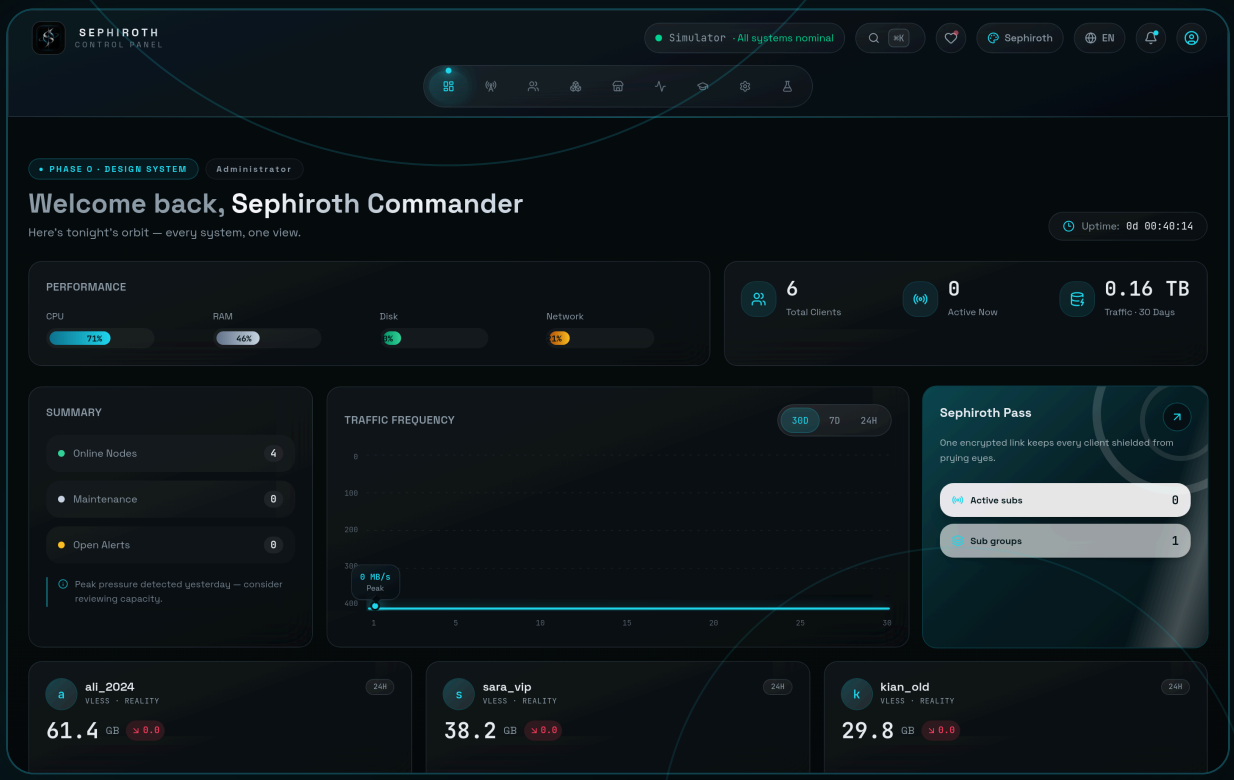


TABLE OF CONTENTS

Table of Contents

01 Introduction — What is Sephiroth? 3

02 Panel Architecture 4

03 Protocols & AmneziaWG 5

04 Forge & Autopilot 7

05 Users & Lifecycle 10

05 Branded Subscription Portal 12

06 Config Doctor 13

07 Real Xray Core 14

08 Security, Alerts & White-label 15

09 Telegram — Alerts & Shop 17

10 Business & Public API 18

11 Deployment — Install, CLI, Docker 20

12 6 Languages · Mobile 22

13 Troubleshooting · FAQ 24

14 Roadmap · Finale 25

◈ About this catalog

Every image in this book is a real panel screenshot (v2.6.0); no private data appears in it.

FORMAT

Vertical PDF, 720×1020

25 pages · 15 real panel screenshots · full English text

AUDIENCE

Server admins, sellers and self-hosters who want a complete Xray panel. Starting from zero? See the Sephiroth Zero-to-Hero Guide (simple Persian).

01 Introduction — What is Sephiroth?

Sephiroth is a complete management panel for **Xray**: inbounds, users and subscriptions, live statistics, security and backups — all in a **single Node.js process**. It needs neither MySQL nor Postgres; data is stored in one atomic JSON file and the UI is served from that very same port.

DESIGN LANGUAGE

Obsidian + glassy cyan

A `#060b0f` backdrop with a `#22d8ee` accent; glassy cards, orbital rings and a starfield — an identity preserved across all 6 themes.

THEMES ×6

6 dark themes

Sephiroth · Jenova · Masamune · Terminal · OLED · One-Winged

LANGUAGES ×6

Persian · English · Arabic · 中文 · Türkçe · Русский

Complete, automatic RTL for Persian and Arabic (dialogs and toasts included); switch language from the top bar, no reload.

ZERO-DEP RUNTIME

No node_modules

In production the whole backend builds into a single-file bundle (`server.cjs`) — ideal for small, single-port hosts.

Sephiroth is built for **personal server admins** who want precise control, for **sellers** who need a reseller system and payment gateways, and for **self-hosters** who want a beautiful, lightweight panel in their own language — with live Socket.IO telemetry, the `Ctrl+K` command palette and an in-app field guide.

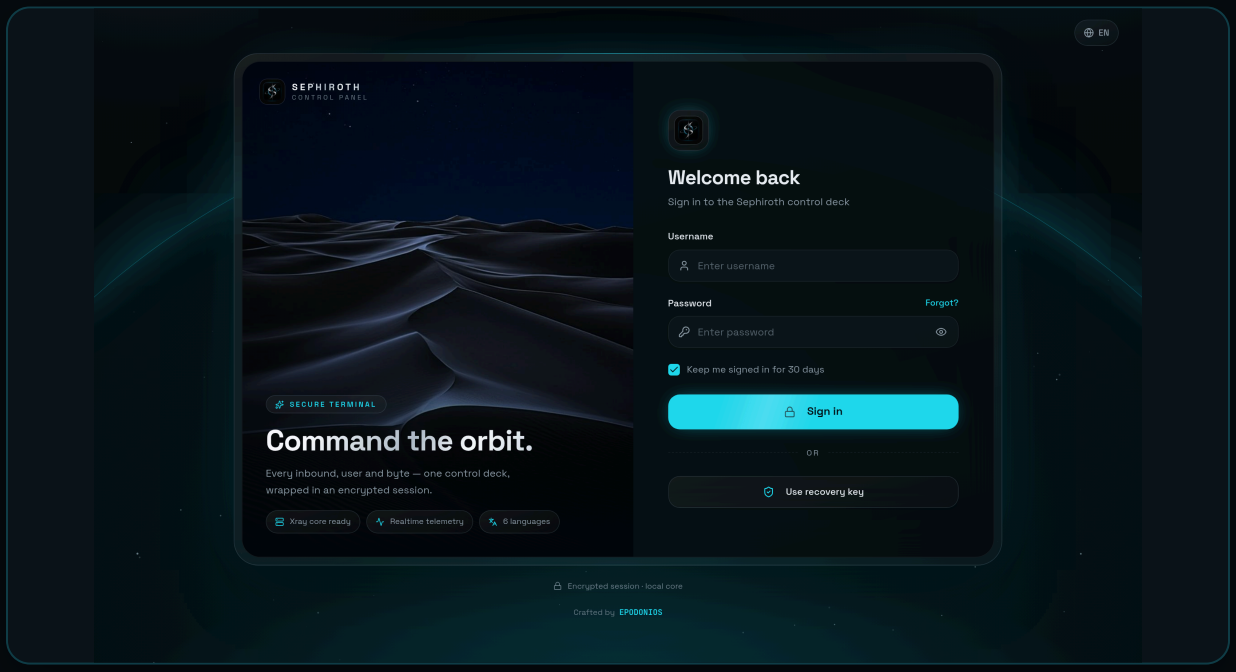
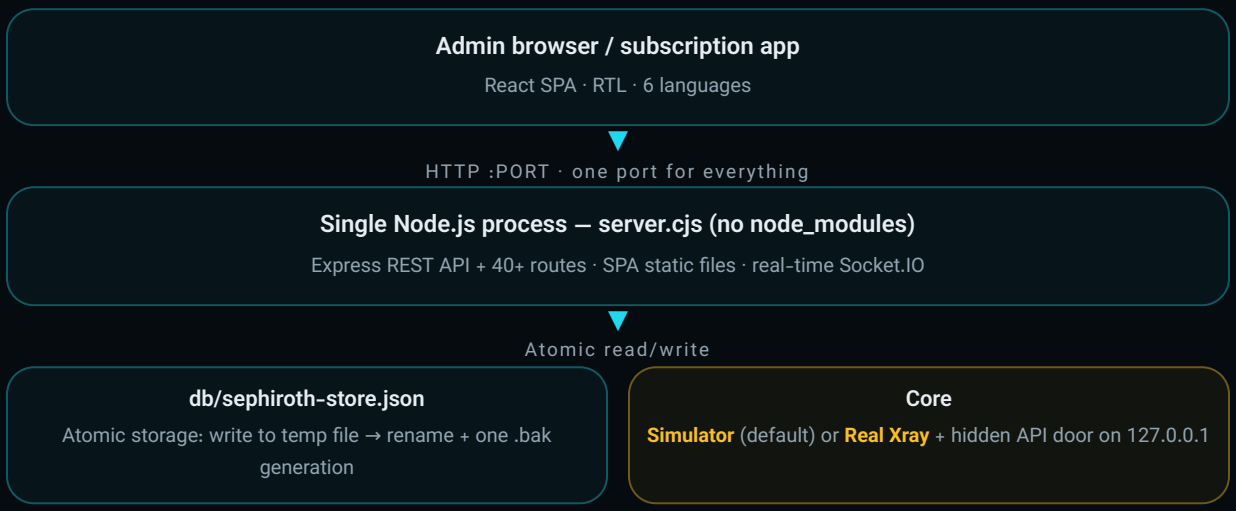


Figure 1 The “Circuit Terminal” login screen — password sign-in, TOTP two-factor login, recovery key and anti-brute-force lockout.

02 Panel Architecture

The whole panel is a **single process**: the Express server (REST API) serves the same port the built React UI (SPA) runs on, while Socket.IO broadcasts live telemetry (CPU/RAM/disk, inbound status, events). The network core has two modes: **Simulator** (default; safe, no binaries) or the **Real Xray driver** (covered in depth in Section 07).



ATOMIC JSON STORE

Every change lands on disk atomically (write to a temp file → rename); a crash costs at most the last operation, never the whole database.

LIVE BY SOCKET.IO

CPU/RAM/disk gauges, per-inbound traffic bars and the activity feed all update via live events — no page refresh.

Deployment options

Method	Good for	Notes
One-line install.sh	Linux VPS/dedicated (Ubuntu/Debian/RHEL/Alpine/Arch)	systemd + `sephiroth` CLI + random port & password
Docker / Compose	Dockerized servers and CI	Official image `ghcr.io/epodonios/sephiroth`
Manual run	Development & testing	`node server.ts` or the `server.cjs` bundle
Single-port hosting	Free application-style hosts (Pterodactyl-like)	Bundle without node_modules + port from `SERVER_PORT`

◈ **Why single-process?**

One port, one process, one data file — on anything that has Node, from a 512 MB VPS to a 308 MB free-host container.

03 Protocols & Transports

The Sephiroth Forge combines **13 inbound protocols** with **6 transports** and **3 security layers**; the subscription link, QR, `.conf` and core output follow automatically.

Protocol	Character & use
VLESS	Modern and lightweight; `xtls-rprx-vision` — the golden pair with REALITY
VMess	The classic, works everywhere; a great fit for CDN and WebSocket
Trojan	Disguises itself as ordinary HTTPS traffic
Shadowsocks	Simple and fast; `2022-blake3` (SS2022) ciphers with key-length validation
Hysteria2	QUIC/UDP-based, for high-latency and lossy networks — independent sidecar
TUIC	QUIC tunnel with BBR congestion control — sing-box engine
WireGuard	The official tunnel with private/public keys and a tunable MTU — sidecar
AmneziaWG	**Censorship-resistant WireGuard with obfuscation parameters — next page**
Mixed	A combined HTTP+SOCKS port on the sing-box engine — perfect for Autopilot
MTProto	Native Telegram proxy via the `mtg` sidecar (auto secret)
SOCKS5 / HTTP	Local proxy with optional auth — the “Telegram proxy” (Section 09)
Dokodemo-door	A transparent forwarding port (also the core's hidden API door)

TRANSPORTS

TCP + header · WebSocket · gRPC · HTTPUpgrade · XHTTP · mKCP

With path/host/serviceName and early-data. mKCP: srtp/utp/wechat-video/dtls/wireguard/dns.

SECURITY

none · TLS 1.0–1.3 · REALITY

ALPN, fingerprint ×9 (chrome/firefox/safari/…), uTLS; REALITY with dest, serverNames, shortIds and an internal keypair.

Fallbacks and the “Single-Port Combo” preset

Every TLS inbound can have a **fallback** (dest/alpn/path) — the **single-port preset**: several protocols on **one port** plus the panel on `/panel` — the complete fix for single-port hosts.

✓ Offline core assembly

Everything you build in the Forge is turned into a real Xray config by the core driver (Section 07) — streamSettings, REALITY, sniffing, mux and fallbacks, with no hand-editing of JSON.

03 AmneziaWG — Censorship-Resistant WireGuard

AmneziaWG is WireGuard with one superpower: hand-crafted junk packets and custom handshake magics that blur WireGuard's fixed signature for DPI. Sephiroth runs it as a **sidecar** (an independent process beside the core): pick the `amnezia-wg` protocol in the Forge, and the panel builds the server keypair and each client's public key, takes the obfuscation parameters (defaults or manual) and hands you ready-to-run server and client `.conf` files.

Parameter	What it means	Default
`Jc`	Number of junk packets sent before every handshake (busywork for DPI)	`4`
`Jmin / Jmax`	Random size range of the junk packets	`40 / 70`
`S1 / S2`	Number of init/response packets padded with meaningless content	`15 / 60`
`H1–H4`	Custom handshake magics (instead of WireGuard's well-known signature)	`auto`
`MTU`	Transmission unit; 1420 by default for an easy ride through the tunnel	`1420`

DELIVERY TO CLIENTS — 2 ROUTES

1) A ready `.conf` file to import into the Amnezia app (each user's own private key, no server-key leakage) — copy/download from the subscription portal and the subscription dialog. **2) A `wireguard://` link** carrying the `jc/jmin/jmax/s1/s2/h1-h4` parameters — v2rayN, NekoBox and Hiddify parse it directly (the "second link" in Autopilot and the Forge).

WHAT HAPPENS SERVER-SIDE?

The panel writes an `awg-quick`-compatible config to `db/core/sidecars/{tag}.conf` (address `172.16.0.1/24`, one `[Peer]` per active client with its own `AllowedIPs`) and, if the binary is present, launches it itself. Binary lookup order: the admin-pinned path → `awg-quick` → `amneziawg-go`.

✓ No binary? You lose nothing

On servers without `awg` installed, the panel keeps working normally and still writes complete server and client `.conf` files; installing `awg-quick` (or setting `sidecarAwgBin` in the settings API) brings the very same config up unchanged — and Config Doctor honestly warns if a WG and an AWG inbound share a private key.

04 Building Inbounds with the Forge

The “Forge” is the inbound create/edit dialog — a smart form that builds the **subscription link and QR live** in the sidebar beside it as you type. The workflow:

1. Pick the **protocol** (9 options; each one is explained right under the field).
2. In the **General** tab: tag, port and listen address — with two magic buttons introduced below.
3. The **Transport** and **Security** tabs: transport + none/TLS/REALITY with every detail (SNI, ALPN, fingerprint, shortId…).
4. In advanced mode: **Fallbacks** as a real array, plus **Sniffing/Mux**.
5. Save → the inbound lands in the card network with its traffic bar and limit chips; add clients right there.

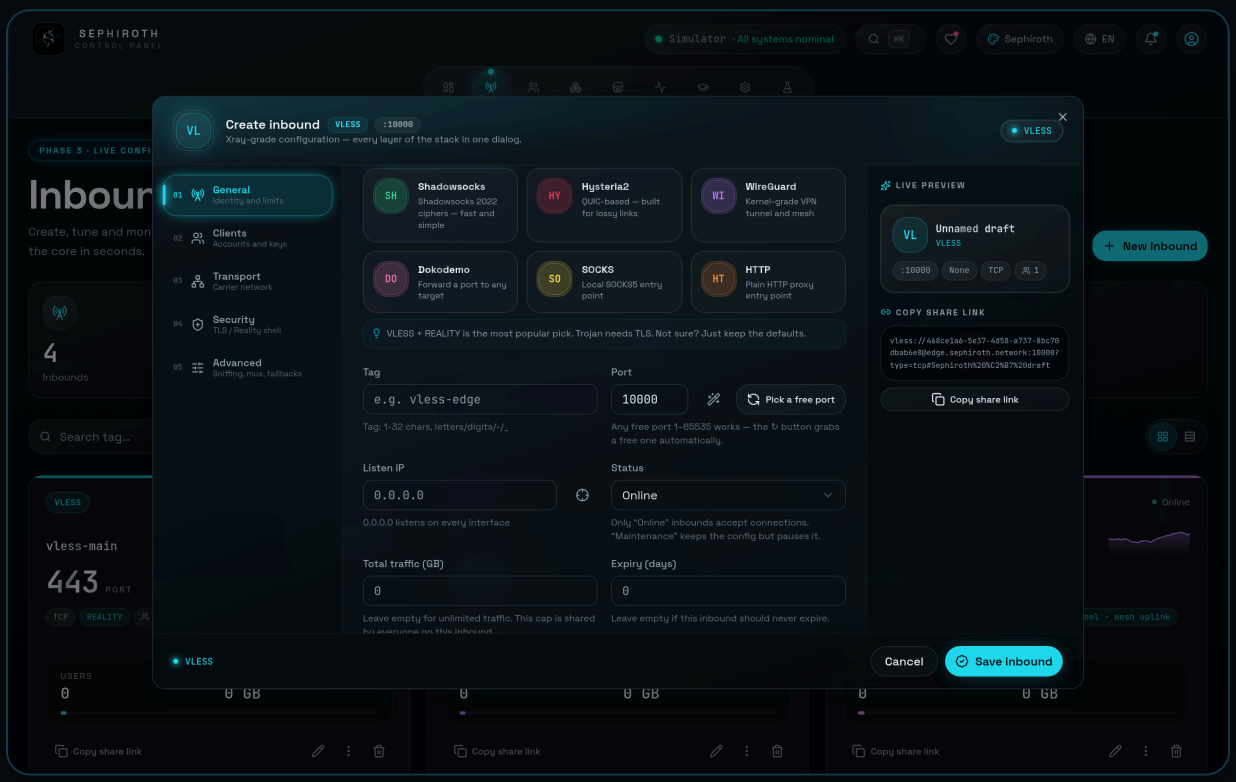


Figure 2 The Forge on the General tab (VLESS protocol) — live subscription-link preview in the sidebar and IP-detect / free-port buttons beside the fields.

Quick Create

If you're not in the mood for forms, hit one of the ready presets in the “Quick Create” grid: `VLESS + REALITY :443` `VMess + WS + TLS` `Local SOCKS5` `Single-Port Combo` and more — it shows a summary of the settings, you confirm, done.

04 Automatic IP & Free-Port Detection

Two small buttons in the Forge eliminate the classic “my link doesn't work” headaches:

✛ SERVER-IP AUTO-DETECT

Next to the “listen” field: a menu with the **detected public IP**, private IPs and `0.0.0.0`. The public IP comes from echo services (cached for 10 minutes) — and more importantly, selecting it switches the **subscription link to the real IP that instant**, so `0.0.0.0` never leaks into a client link.

✦ FREE-PORT WAND

Next to the “port” field: a wand that suggests the **first free port** (panel port +1 and +2, then 8443/443/2053/2087/2096/8880). No more “port in use” errors or duplicate ports.

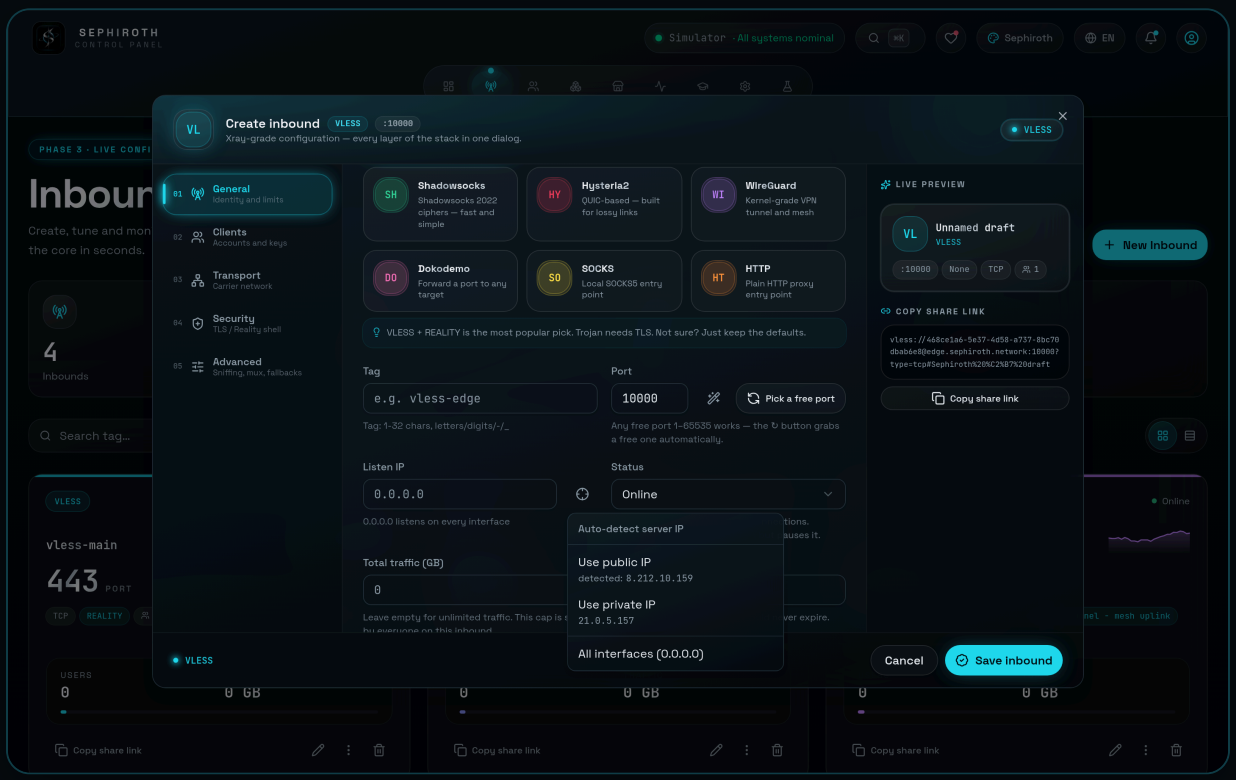


Figure 3 The IP-detection menu is open: “Use public IP — detected”, private IPs and all interfaces — real values from the internal `/api/system/server-info` API.

> **⚠ Pro tip**
>
> If you sit behind a CDN or reverse proxy, set “listen” to `0.0.0.0` or a private IP; only the public IP/domain lands in client links — the Forge keeps those two concepts apart.

04 Autopilot — a Config Fleet in One Click

Instead of building inbounds one by one, the **Autopilot** wizard assembles a multi-protocol fleet in three steps — with automatic port allocation, template inheritance and the subscription link in the output. The “Autopilot” button sits beside “New config” in the inbounds view.

1. **Source:** “mirror an existing inbound” (a full clone of its transport/security/template users) or “pick several protocols” (fresh inbounds with fresh keys).
2. **Build plan:** a table of the chosen protocols with an editable port, the run engine and a users column; the TLS switch, “start port” and “contiguous allocation” live right here.
3. **Build:** one atomic POST; rows get their checkmark on the spot and a report with each inbound's subscription link arrives at the end.



Figure 4 Autopilot step 2: automatic ports, the per-protocol engine (Xray / sing-box / sidecar) and honest “already exists” warnings.

PORT ALLOCATOR

Preferred public ports first (`443, 8443, 2053, 2083, 2087`), then contiguous ones; occupied and duplicate ports are skipped automatically.

ENGINE MAP

VLESS/VMess/Trojan/SS → the Xray stream core · Hysteria2/TUIC/Mixed → the sing-box engine · MTProto → the `mtg` sidecar · WireGuard/AWG → the WireGuard sidecar.

USERS

In multi-protocol mode: “one fresh user” per inbound or “one shared identity” for all — in clone mode, the template's users are copied.

◈ Atomic and rollback-safe

If any protocol fails midway through the build, everything built so far is torn down and the ports stay free; in the report, each row's subscription link + “second link” (the AmneziaWG `.conf` file) is copied with one click.

05 Users & Subscriptions

Every user is a **real subscriber**: traffic quota, expiry day, IP limit and several simultaneous inbounds. The users table shows a live usage bar, active/expired status and smart chips:

- **Usage-forecast chip:** "≈4 days to cap · 3.2 GB/day" — the burn rate comes from dividing usage by account age (no history table needed); amber under 7 days, red under 3.
- **Reset cycle:** weekly/monthly quotas reset automatically via a built-in cron, and expired users are deactivated at the same time.
- **On-hold and auto-renew:** the subscription clock starts at the user's **first connection**, not at the moment the account is created; and with auto-renew, the window refreshes at expiry and keeps serving instead of being cut — details on the "Lifecycle" page.
- **Device (HWID) and IP limits:** each subscription's device count is enforced in the sub portal, and the IP limit (optional, fail2ban-style) auto-cuts over-limit users at the same time.
- **Inbound access:** a user can ride multiple inbounds; each mount's key (UUID/password/flow) is managed right there.

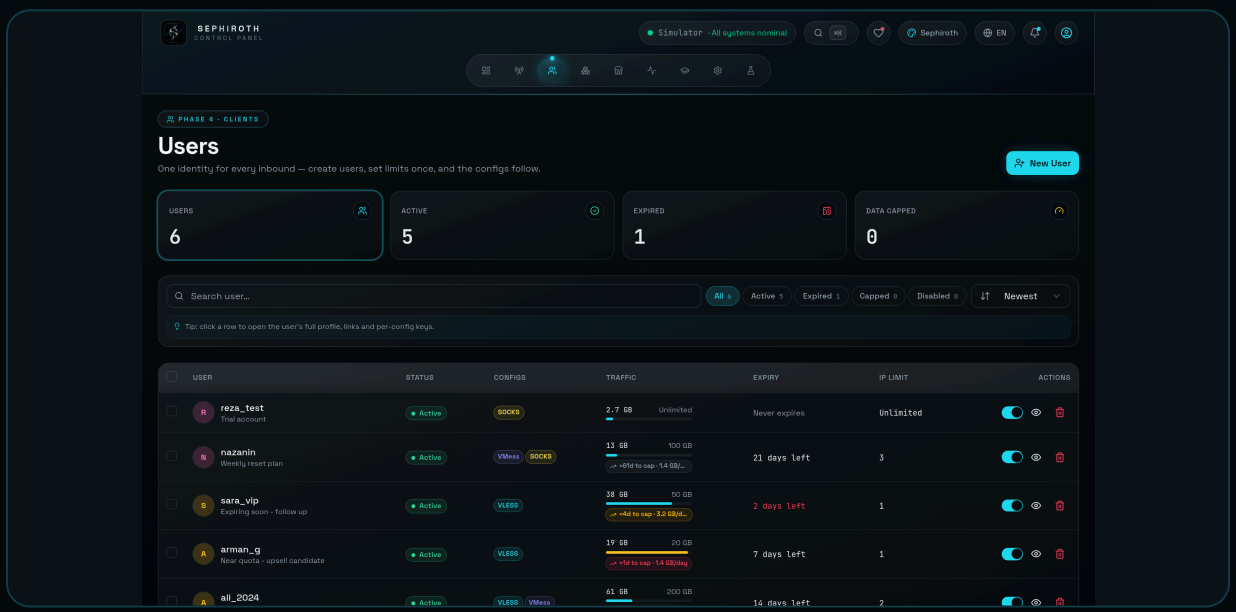


Figure 5 The users table with usage bars, quota/expiry and "≈N days to cap" forecast chips.

◈ Real limit enforcement

The IP limit (`0 = ∞`) is recorded in links and in every client's core output, and the "enforce IP limit" switch in the Core card auto-disables over-limit users in 10-minute windows (off = warn only). The HWID device limit is enforced in the subscription portal — the 13th device sees a "device limit reached" page, and the admin frees it via "Reset devices".

05 User Lifecycle — On-Hold, Auto-Renew, Bulk Actions

Sephiroth manages a subscription's life like a real sale: an account can stay **un-counted until its first connection**, **renews itself** at expiry instead of being cut, and dozens of users are managed in one click.

ON HOLD

With the "on hold (starts on first connect)" switch, the account is created as **on hold, starts on first connect**: no countdown is shown anywhere and the expiry day locks at the user's **first real connection** (first sub fetch or first traffic). Switching it back to "on hold" clears the anchor too, so the count restarts at the next connection.

AUTO-RENEW

With the "auto-renew" switch, at the moment of expiry the panel's built-in cron opens a **fresh full window** from that point, zeroes usage, bumps the "renewals" counter and — instead of deactivating — keeps the subscription alive: the "rolling subscription" pattern with zero admin work.

Bulk actions — 11 actions on any number of users

Action	Effect	Action	Effect
Enable / Disable	Account status	Renew	A fresh full window from now + usage reset
Reset traffic	Usage → 0	On hold / Off hold	The first-connect anchor
Auto-renew on/off	The rolling switch	Reset devices	Clear the HWID registrations
Adjust limits	Days/volume/IP — an empty field means "leave untouched"	Delete	With a confirmation dialog

- **Bulk create:** a username prefix + count (up to 100) with the same limits; **user import/export** as JSON is there too.
- **Devices (HWID):** each user's drawer lists the registered devices (UA + last seen) with a "Reset devices" button — the 13th device stops in the portal on a "device limit reached" page.

✓ Everything notifies

First-connect activation of an on-hold account, auto-renew, near-expiry and failed login — all of these are notification-engine events (Section 08): the top-bar bell + the admin's Telegram.

05 The Branded Subscription Portal

Every user has a personal subscription link: `/sub/<subId>`. The subscription page is a branded “orbit pass” — a usage bar, days remaining and one-click import into popular apps. Sub groups also bundle several users behind **one password-locked link**. Page branding (name/logo/accent) is applied automatically from panel settings and from each reseller's profile — full white-label, down to the footer signature.

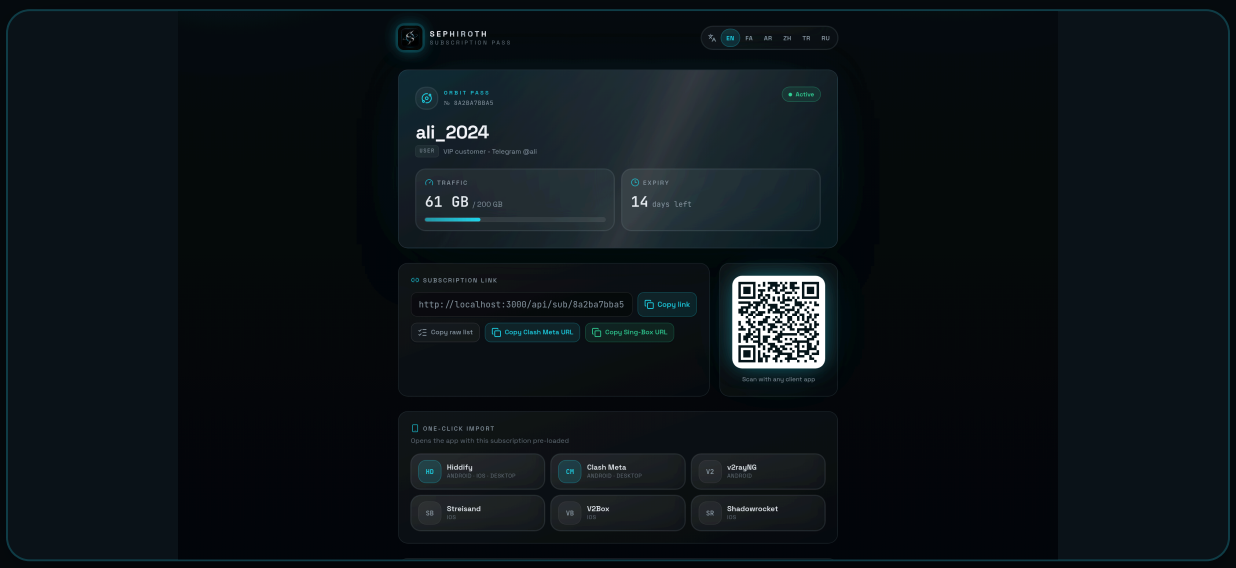


Figure 6 A user's subscription portal (ORBIT PASS): 61 of 200 GB quota, 14 days remaining and one-click import — Sephiroth-branded with the EPODONIOS signature.

Subscription output formats

Format	Fetched with	Good for
Raw URI	`?format=raw`	Manual sharing
Base64	`?format=b64`	Most v2ray apps
Clash Meta	`?format=clash`	Clash / Mihomo / Verge / Stash
Sing-Box	`?format=singbox`	Sing-Box 1.10+ / Hiddify / Streisand / V2Box
Xray-JSON	`?format=xray`	Full v2ray/xray-core profile

✓ Automatic User-Agent detection

The client app identifies itself: Clash/Mihomo/Verge/Stash get `YAML` output, sing-box/Hiddify/V2Box get `sing-box` output, and everything else gets Base64 — with zero configuration on the user's side.

06 Config Doctor

Config Doctor is a **unique Sephiroth feature**: a 16-point scan of the whole panel that finds problems before users discover them and gives every finding “title + explanation + fix”. The score is computed out of 100:

```
score = 100 − 12×danger − 5×warning − 1×notice  →  grade A to D
```

Inbounds (8 checks)

Open SOCKS/HTTP proxy without auth · TLS without a certificate · incomplete REALITY (dest/serverNames/shortIds/keys) · broken fallback · WireGuard private-key reuse · privileged port (<1024) · expired inbound · disabled inbound

Users (6 checks)

Expired but still active · near expiry (≤3 days) · exhausted quota · ≥90% usage · a “ghost” account with access to no inbound · a user mounted on a disabled inbound

Routing (2 checks)

A dead rule (nonexistent outbound) and an empty, conditionless rule — the ones that silently swallow traffic or reject everything.

Scan output

A score ring + grade + severity chips (danger/warning/notice) and, for every finding, a precise explanation and a suggested-fix button. All strings are translated into 6 languages.

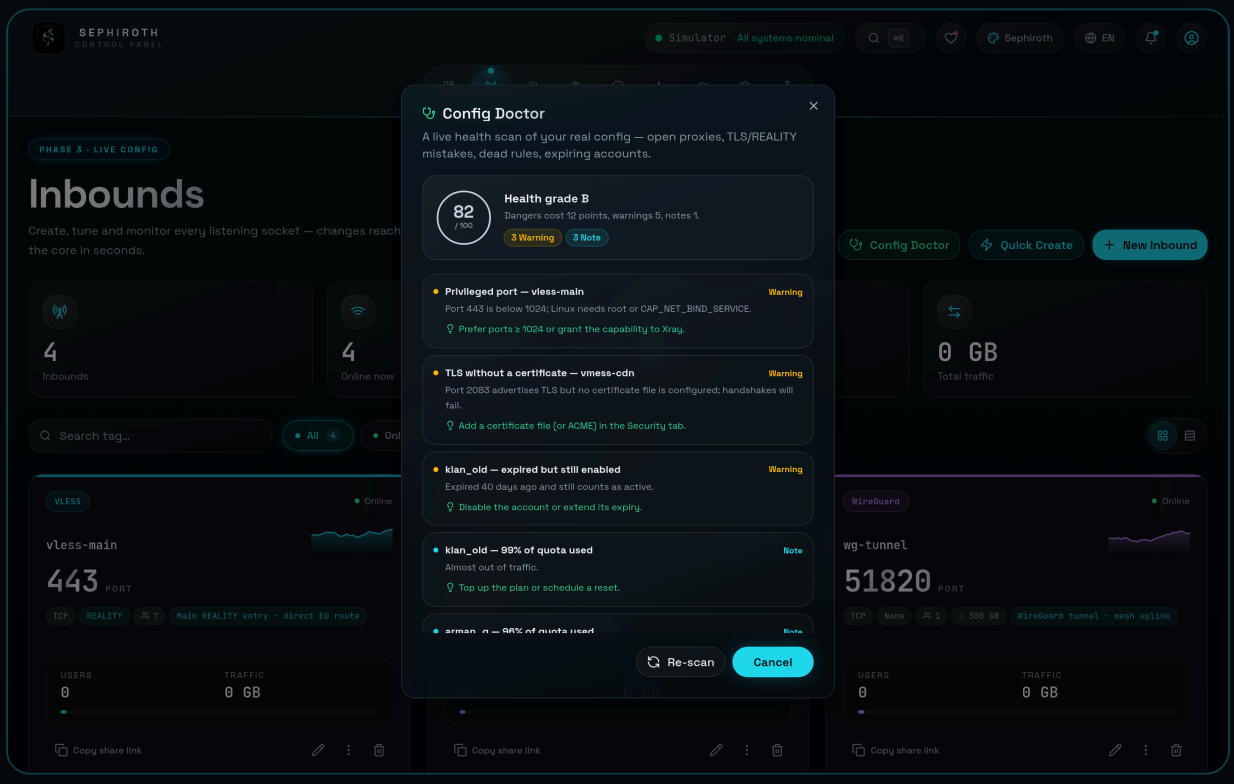


Figure 7 Config Doctor on a real panel: score 82 (grade B) with real findings — privileged port 443, TLS without a certificate, and an expired user.

07 Real Xray Core

As of v2.4 Sephiroth ships **two core drivers**. The default is the **Simulator** (safe, display-only telemetry, no binaries — for any hosting including locked-down hosts), while the **Real Xray driver** runs the actual core as an opt-in:

Capability	Simulator (default)	Real Xray (opt-in)
Protocol execution	Simulated telemetry	**Real execution — traffic passes through the real port**
Per-user stats	Display-only	**Real (stats API deltas, every 10 s by default, tunable 5–120)**
Requirements	None	Automatic binary download from XTLS/Xray-core releases (or a manual path + version pin)
Config validation	—	`xray run -test` before every run
Good for	Testing, demos, restricted hosts	**Production**

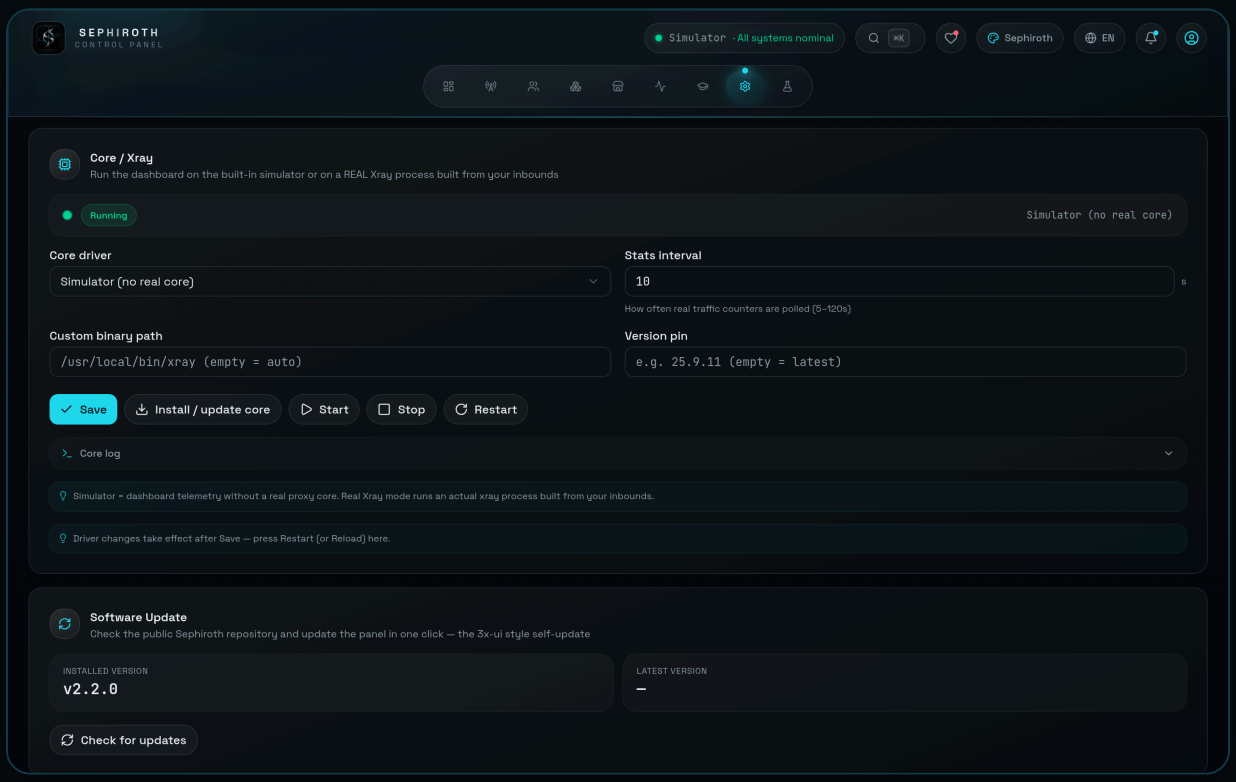


Figure 8 The "Core / Xray" card in Settings: driver selection, install/start/stop/restart, binary path, version pin and a collapsible live log.

- **Build from the panel:** the config is generated from the panel's own inbounds/users; the hidden stats API door comes up on `127.0.0.1`.
- **Reload on change:** every inbound/user change applies after a ~1.5 s delay; the core restarts only if the config hash actually changed.

08 Security

TOTP two-factor login

A zero-dependency RFC-6238 implementation (SHA-1, 6 digits, ±1 window). Enrollment with QR + a confirmation code; **disabling it asks for a code too**, so a stolen session can't switch 2FA off. After the password, the login page asks for the code.

Anti-brute-force

5 failed attempts in 5 minutes = the IP gets locked, with a live countdown on the login screen; the full attempt log lives under “Activity”.

Recovery key

The `SEPH-XXXX-XXXX-XXXX` key is shown only once and can be rotated whenever you like — forgetting the password becomes a simple sign-in, not a lost panel.

Sessions

An httpOnly cookie signed with HMAC; “revoke all sessions” rotates the secret and invalidates every cookie. The `SEPH_COOKIE_SECURE=auto` cookie mode detects on its own whether you're behind HTTPS.

Scheduled backup

Every 24 hours a full JSON export lands in `db/backups/`, keeping 1–30 files (oldest pruned) + immediate run, download and deletion from the UI. Manual backup/restore with `RESTORE` confirmation is there too.

Software update

A “check → install → restart” button inside the panel: the updater runs as a separate process, publishes its status to a status file, and the panel keeps serving the previous code until restart. Non-git installs update via the CLI.

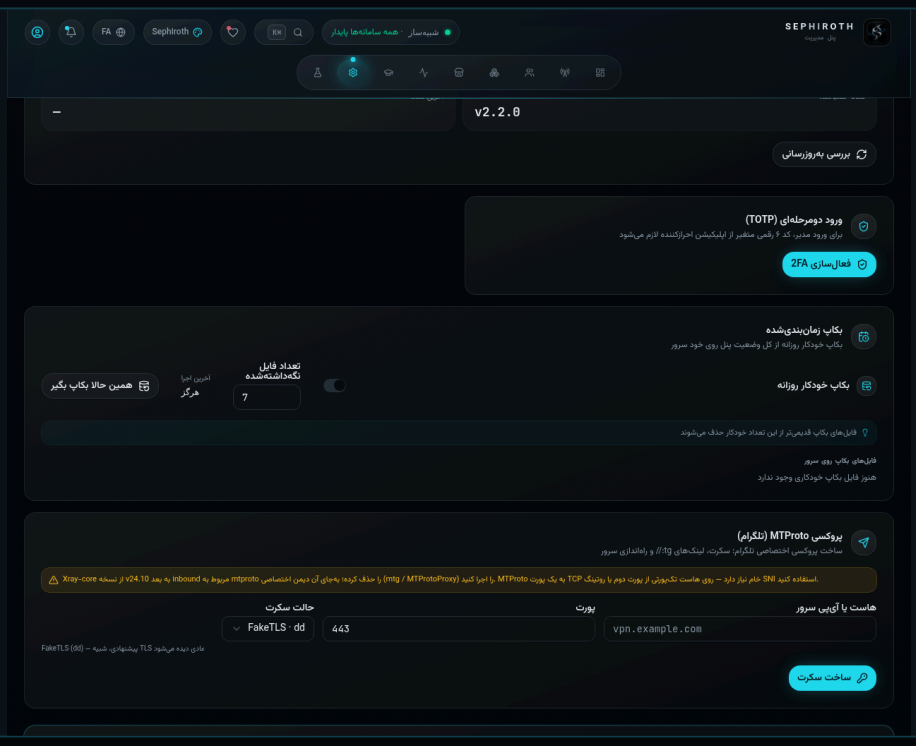


Figure 9 The security/system cluster in Settings (Persian RTL): software update, two-factor login, scheduled backup and the MTProto tool.

08 Notifications & White-label

The notification engine pushes important events — smartly deduplicated — to the **top-bar bell** and the **admin Telegram bot**; every kind has its own switch in Settings.

Event	When does it fire?
New / deleted / edited user	Every user creation, deletion and change (name, limits…)
Expiry & near-expiry	Account expiry + last-days warning (max one ping per user per day)
Quota & periodic reset	A filled quota and the weekly/monthly reset
Activation (on-hold)	The on-hold user's first connection — the subscription clock locks here
Auto-renew	The window rolling over at expiry + the usage reset
Failed login	Failed attempts with IP — max one per user+IP per 10 minutes
Daily sales digest	Daily cron: completed orders, revenue (Toman), new users — min 23h gap
System	Backup done, panel restart, inbound created/deleted

Top-bar bell

The last 8 events with icon and relative time; a pulsing dot until opened; a link to open the 500-event activity log for the full history. The “seen” state is local — opening the panel marks everything as seen.

Telegram delivery

Separate switches in Settings (new users, user lifecycle, traffic, failed login, daily digest) + “send test message” and “send test digest” buttons to see it right now, without waiting for the cron.

White-label — your panel, your brand

Panel brand

Settings → General → “Subscription branding”: brand name, logo URL, accent color + “clear brand”. Every user without a dedicated brand renders their subscription page with this identity — down to the footer signature.

Reseller brand

In each reseller's dialog: a dedicated brand name, logo and color. That reseller's users see their brand in the sub portal; without one, the panel brand applies — a complete hierarchy with zero extra setup.

✓ Honest

Notifications are sent only when the bot is enabled and its mode isn't “console”; the token and the admin chat ID must be set as well — the test buttons verify exactly that for you immediately.

09 Telegram

The eternal question — “how do I get Telegram past censorship?” — is answered inside the panel itself, with a complete suite and some honest notes:

1) The SOCKS5 proxy suite

When you build a **SOCKS5** inbound in the Forge, the “Telegram proxy” block opens and, alongside the standard link, gives you two native Telegram links + a QR: `t.me/socks?server=…&port=…&user=…&pass=…` and `tg://socks?…` — the user taps once and Telegram accepts the proxy.

2) The MTProto tool (Settings → System)

For more hostile networks, the built-in “MTProto proxy” tool generates a real secret: you choose the host, port and `dd` (domain camouflage) or `plain` mode, receive the secret + `tg://proxy` and `t.me/proxy` links + QR, and the ready-to-run `mtg v2` (Docker) commands sit in the same card.

> **⚠ Technical honesty — things you should know**
>
> 1) SOCKS5 in Telegram has **no camouflage** and is visible to DPI; use MTProto on restrictive networks. 2) Xray itself removed the MTProto inbound in v24.10 — instead of a false claim, Sephiroth ships the independent `mtg` sidecar (Section 03) + the secret-generation tool.

3) The Telegram shop — bot modes

The Sephiroth bot has three modes: **Console** (test runs only), **Notifications** (a worker for alerts) and **Sales shop** — full selling in Telegram: the user picks a language, sees the sale templates, pays via **ZarinPal** or sends a **card-to-card** receipt; the admin approves in chat and the order is provisioned instantly — the `/sub/` link is delivered automatically.

Bot gateways

ZarinPal v4 (request / auto-approve) · card-to-card with receipt-photo approval in chat · per-template pricing (`priceToman`) · bot texts in 6 languages.

Bot notifications

New orders + panel restart to the admin · completed order to the customer · the worker state as a live pill (running/stopped) with the last poll.



Figure 10 Bot notification settings + the “Bot mode” card: Console / Notifications / Sales shop, the sale template, payment method and the worker status pill.

10 Business — Resellers & Gateways

Sephiroth is the only panel in the comparison lists with a **full reseller system** — something 3x-ui/Marzban/Remnawave don't have:

Resellers

- **A second role with independent auth:** a reseller signs in with their own account and the navigation only shows “Users”.
- **Hard quotas:** caps on user count and allocated traffic — quota violations are rejected at the API level (not just the UI) with a `403`.
- **Scoped lists:** a reseller sees only their own users, and even hand-tampered requests lose other users' records to a `404` (without revealing that they exist).
- **Per-reseller brand (white-label):** brand name, logo URL and accent color in the reseller dialog — that reseller's users see their brand on subscription pages (empty = panel brand).
- **Admin monitoring bars:** for every reseller you see how many users out of how many quotas, and how many GB out of how many GB, have been consumed.

Sale Templates

Ready-made packages (“Starter 50G”, “Pro 200G”, …) that appear as one-click chips in the user-creation dialog and fill quota/expiry/IP-limit in one go — sellers build the right package without thinking about the numbers.

Payment gateways

Gateway	Connection type
ZarinPal / IDPay	Webhook — automatic recharge of reseller quota
Crypto USDT (TRC20)	Crypto payment configured right in the panel
Telegram card-to-card	Card-to-card payment via Telegram

> ◈ **Usage forecasting = sales management**
>
> The “≈N days to cap” chips matter for resellers too: before a subscriber's quota runs out, you already know which accounts will want a renewal tomorrow — prepare the invoice before the outage.

10 Public API — Marzban-Style Automation

The whole panel is programmable from the outside: a stable `/api/v1` namespace with bearer `seph_…` tokens, official **OpenAPI 3.1** specs at `/api/openapi.json` and an **interactive Swagger UI** at `/api-docs` — hit Authorize with your token and test right from that page.

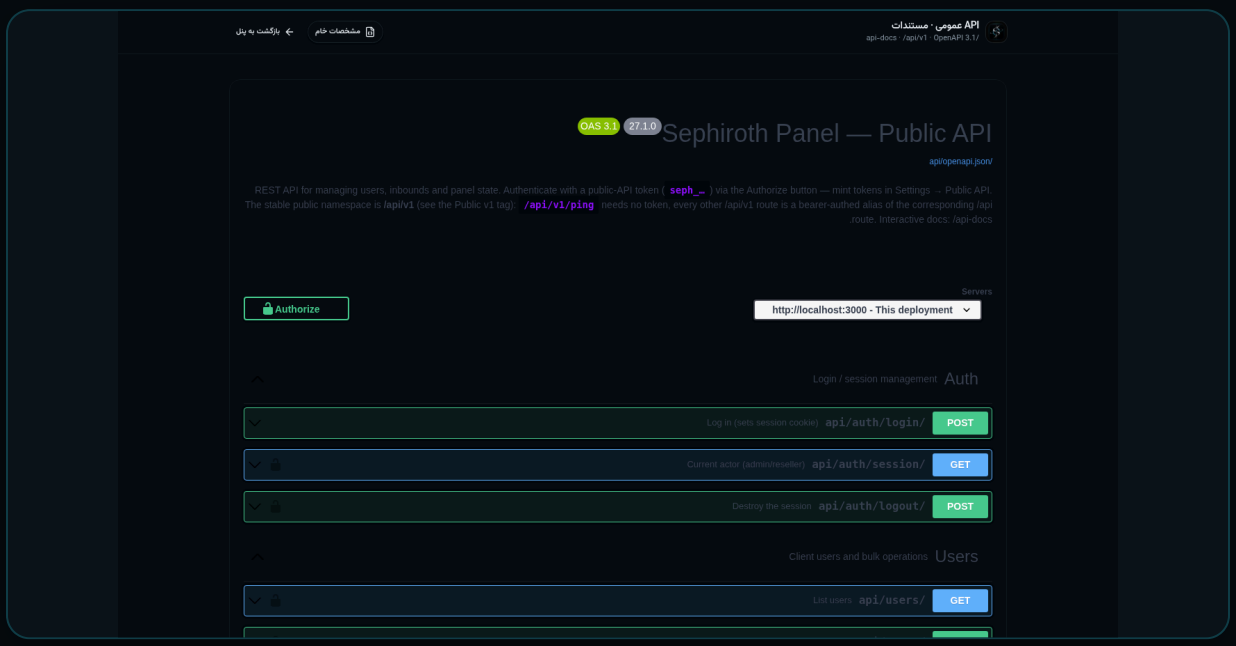


Figure 11 The standalone `/api-docs` page: OAS 3.1, the Auth/Users/System/Public v1 tags and the seph_ token authorizer.

Group	Routes
Ping (no token)	`GET /api/v1/ping`
Users	`/api/v1/users` CRUD + `bulk` (11 actions) + `bulk-create` + `import/export` + `ip-usage` + `reset-devices`
Inbounds	`/api/v1/inbounds` CRUD for the 13 protocols + `autopilot` (clone/multi)
System	`/api/v1/system/server-info · core-status`
Activity	`/api/v1/activity`

TOKENS

Minted in Settings → Public API; the secret is displayed exactly once (only its sha256 hash is stored), “last used” with IP and time, revoke and on/off.

BEARER EVERYWHERE

Besides `/api/v1`, every dashboard REST route also accepts an `Authorization: Bearer seph_…` header beside the session cookie.

ERRORS

Meaningful codes: 401 without a token, 423 “device limit reached” on subscriptions, 403 reseller quota — ready for scripts and monitors.

11 Deployment — One-Line Install & CLI

On any Linux server (Debian/Ubuntu/RHEL/Alpine/Arch, x86_64 and arm64) it's just one line:

```
# Interactive install (asks for port / user / password)
bash <(curl -Ls https://raw.githubusercontent.com/Epodonios/Sephiroth/main/install.sh)

# Install with a pinned version
bash <(curl -Ls https://raw.githubusercontent.com/Epodonios/Sephiroth/main/install.sh)
v2.6.0

# Unattended install (CI/scripts) — output: /etc/sephiroth/install-result.env
SEPH_NONINTERACTIVE=1 bash install.sh
```

What does the script create?

- `/opt/sephiroth` — the source + SPA build (vite) and the single-file `server.cjs` bundle (esbuild); then node_modules is wiped → a zero-dependency run.
- `/etc/sephiroth/sephiroth.env` — a random free port (20580–21000), a **random 16-character password** and `SEPH_COOKIE_SECURE=auto` (re-running preserves existing values).
- **A hardened systemd service:** NoNewPrivileges, ProtectSystem=full, PrivateTmp — and on hosts without systemd, a daemon mode via the CLI.
- **Install summary:** the panel address and credentials are printed and saved to `install-result.env` with 0600 permissions. Node 22 is installed too if missing (NodeSource → distro repository → pinned official tarball).

Management CLI — `sephiroth`

```
$ sephiroth          ← interactive menu
  1. Start               6.  Logs (follow)
  2. Stop                7.  Logs (last 100)
  3. Restart             8.  Change Admin Password
  4. Force Restart       9.  Change Panel Port
  5. Status              10. Backup Now
  0. Exit                11. Restore Backup
                         12. Check for Update
                         13. Update Panel
                         14. Uninstall
# or directly: sephiroth start|stop|restart|status|logs|backup|update …
```

Password/port changes go through the maint module (scrypt hashing in the panel's own format), backup/restore matches the in-panel backup format, and “check for update” compares the version against the public repo's package.json.

11 Deployment — Docker & Updates

The official multi-stage image builds on `node:22-alpine`: stage one builds the SPA + esbuild bundle, stage two ships only `server.cjs` + `dist/` — no source, no node_modules. A built-in healthcheck hits `/api/auth/session` and the `/app/db` volume keeps data persistent.

```
# Fastest way
docker run -d --name sephiroth -p 3000:3000 -v sephiroth_data:/app/db \
  ghcr.io/epodonios/sephiroth:latest

# docker-compose.yml
services:
  sephiroth:
    image: ghcr.io/epodonios/sephiroth:latest   # or pin a version: :v2.6.0
    ports: [ "${SEPH_PORT:-3000}:3000" ]
    environment:
      SEPHIROTH_ADMIN_USER: ${SEPHIROTH_ADMIN_USER:-admin}
      SEPHIROTH_ADMIN_PASS: ${SEPHIROTH_ADMIN_PASS:-sephiroth}  # change it after first
login!
      SEPH_COOKIE_SECURE: auto
    volumes: [ sephiroth_data:/app/db ]
```

Tags are built automatically on GitHub Actions and pushed to GHCR: `latest` for main plus semantic version tags (`v2.6.0`) — pinning the version in production is recommended. The env username/password are only seeded on the **first boot with empty data**; after that, change them only from inside the panel or the CLI.

Updating (script installs)

Re-running install.sh = a safe update: the database, env and passwords stay untouched and the service restarts. The panel itself has an update button too (Section 08). Non-git mode: download the zip + swap `server.cjs`.

Complete removal

`sephiroth uninstall` with two confirmations: the service and files are removed; data (db) removal is optional and asked separately.

⚠ Post-install checklist

1) Change the default password (Profile → Security). 2) Store the recovery key somewhere safe. 3) Enable 2FA. 4) Take a manual backup so you've practiced a restore once. 5) In production, enable the Real Xray driver from the “Core” card.

12 The Panel in 6 Languages

The panel UI, subscription portal, toasts, dialogs and even the Config Doctor strings are translated into **6 languages** in full parity (automated check: zero missing keys and zero placeholder mismatches across all 6):

Language	Code	Direction	Note
Persian	`fa`	RTL	Standard ZWNJ half-spaces and Persian digits
Arabic	`ar`	RTL	Consistent terminology (entry points, distributor…)
English	`en`	LTR	The reference language
中文	`zh`	LTR	Full-width parentheses and fluent Chinese phrasing
Türkçe	`tr`	LTR	Consistent terminology (Giriş / Sorgula…)
Русский	`ru`	LTR	Localized technical terms

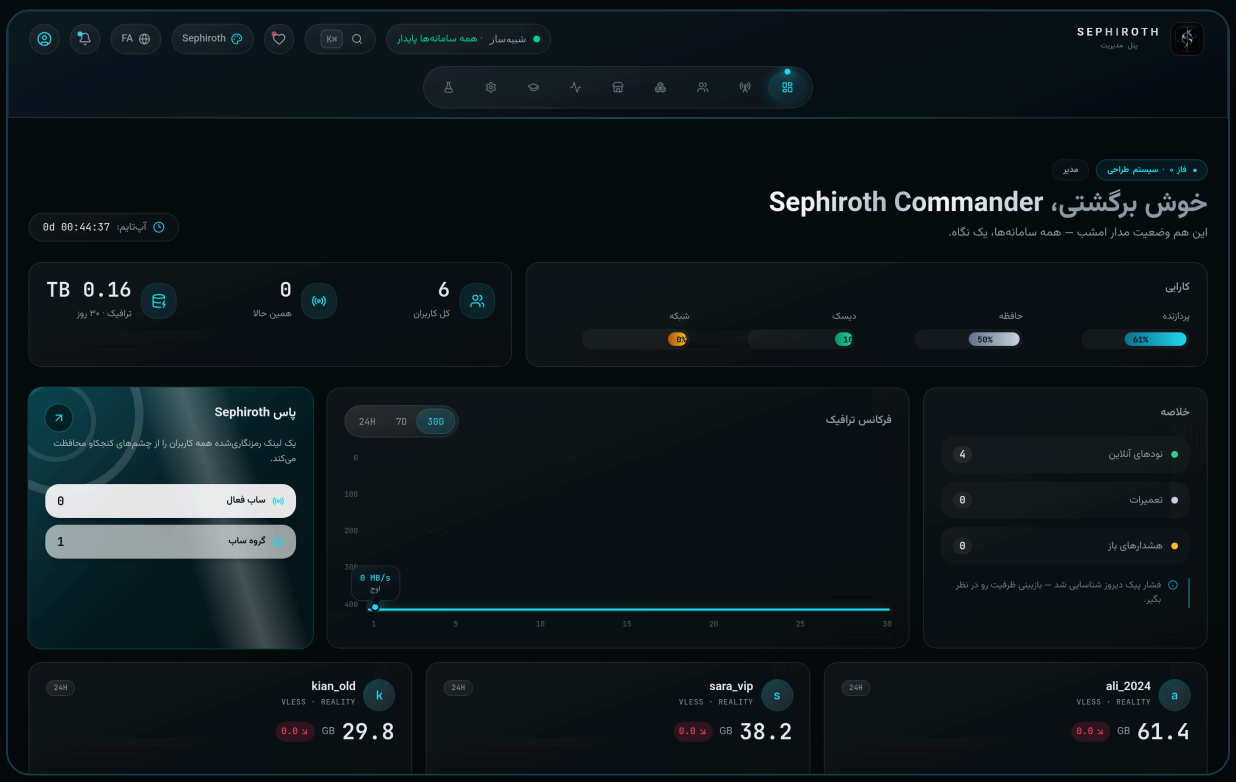


Figure 12 The dashboard in Persian — complete RTL mirroring: gauges, charts, navigation and the live feed.

◈ Real RTL, not just right-aligned

Page direction is injected at the root via the Radix DirectionProvider — dialogs, dropdowns, tooltips and even toasts get mirrored. Technical numbers (ports, versions, links) stay inside `dir=ltr` blocks so they remain readable.

12 Mobile & Responsive

All 13 panel views are redesigned down to 390 px width too: tables become stacked cards, the top bar collapses and large dialogs (like the Forge) switch to vertical scrolling with full-width fields — in both LTR and RTL directions, with no horizontal overflow (tested with automated layout inspection).



Figure 13 Mobile dashboard (English)



Figure 14 Mobile Forge — Persian RTL

MOBILE-FIRST TOUCHES

Global search and the `Ctrl+K` command palette work on mobile too; move between views via the same top navigation and the dashboard's quick-action buttons.

QA AUTOMATICO

A layout auditor (overflow, overlap, text clipping) scans every view in EN/FA at 1280 px/390 px — the same audit this catalog passed.

13 Troubleshooting · FAQ

Question	Short answer
I get “port already in use”	Hit the ✦ wand button in the Forge to get the first free port suggested; to check manually: `ss -tlnp | grep <port>`
The Xray binary download fails	Set the binary path manually or pin a version in the “Core” card; until then the panel keeps working normally with the Simulator.
Do Simulator traffic numbers differ from real ones?	Yes — the Simulator's telemetry is for display. Real per-user stats come only from the Real Xray driver (Section 07).
I forgot the admin password	From the server: `sephiroth` → option 8 (change password); or use the “recovery key” on the login page.
How do I restore a backup?	Settings → Restore → JSON file + `RESTORE` confirmation; or on the server: `sephiroth` → option 11. The restore runs in one transaction and the admin account stays untouched.
How do I enable HTTPS?	Put a reverse proxy (Nginx/Caddy) with TLS in front; the secure cookie has an `auto` mode and turns Secure by itself via `x-forwarded-proto`.
I only have one port (free hosting)	Build the “Single-Port Combo” preset (Section 03): several protocols with fallback on one port + the panel on `/panel`.
After a change, the panel still runs old code	That's normal — the in-panel update waits for “restart” so sessions don't drop; press the restart button.
The AmneziaWG inbound won't come up	The `awg` binary isn't on the server. The panel always writes the server-side and client `.conf` files; installing `awg-quick` or pinning the binary path (setting `sidecarAwgBin`) brings the same config up unchanged.
The daily sales digest doesn't arrive	Enable it in Settings → Telegram bot and set the token + admin chat; the “Send test digest” button sends it right now, without waiting for the cron.

✓ Quick troubleshooting toolbox

`sephiroth logs` for service logs · `sephiroth status` for status · the live core log in the “Core / Xray” card · the Config Doctor report to clean up your config · the “Activity” feed to see who did what.

14 Roadmap · Finale

Delivered through v2.6.0

✓ Forge with REALITY · ✓ Config Doctor · ✓ Real Xray driver + sing-box · ✓ 2FA · ✓ Scheduled backups · ✓ In-panel updates · ✓ Telegram shop (ZarinPal/card-to-card) · ✓ Multi-node fleet · ✓ In-panel ACME · ✓ Autopilot · ✓ AmneziaWG (sidecar) · ✓ On-hold + auto-renew · ✓ Notification engine + bell · ✓ Panel/reseller white-label · ✓ 11 bulk actions · ✓ Public API `/api/v1` + Swagger · ✓ 5 subscription formats + branded portal · ✓ 6 themes · ✓ 6 languages · ✓ Install suite · ✓ IP/port auto-detect

Ahead

- **Group branding** — a dedicated brand for each sub group (besides panel and reseller).
- **Fleet scheduler v2** — periodic auto-sync of inbounds across servers.
- **Event webhooks** — push panel events to any custom URL.
- **IP-limit enforcement on remote nodes** — extending the IP guard to the whole fleet.

SEPHIROTH · v2.6.0

Sephiroth is an **open-source project under the MIT license** — fork it, modify it and use it in your own projects. If it was useful to you, encourage us with the in-panel “Support Sephiroth” button or a GitHub star.

Disclaimer: this panel is published for personal and educational use; responsibility for any misuse lies with the user. All images in this catalog were taken from the real v2.6.0 and contain no private data.

Built with ♥ by **EPODONIOS** · github.com/Epodonios/Sephiroth